

Operating Systems
Lab 7

1. Write the following C program. A parent forks a child process, the parent executes inside a forever loop. The child sleeps for 5 seconds to give time to the parent to enter a forever loop. The child then sends the signal SIGINT to the parent. After the parent receives the signal, it prints "Get SIGINT from child", exit the forever loop, print "Bye", and terminates.

The example output is as follows:

```
parent: Waiting for my child to send SIGINT
child: Sleeping for 5 seconds
child: sending SIGINT
child: Bye
get SIGINT from child
parent: Bye
```

Note: Use system call kill() send signal from child to parent. To retrieve the parent process id use the system call getppid().

2. Write the following C program. A parent forks 5 children processes. Each child prints its own process id and enters a forever loop. Parent sleeps for 2 seconds to give time to all children to enter their forever loop. Parent then sends SIGTERM to all children. When each child process receives the SIGTERM, it prints its process id, and terminates. After all children have been killed, the parent prints "Parent: I have killed all of my children". The example output is as follows:

```
Child: my id is 430
Child: my id is 431
Child: my id is 432
Child: my id is 433
Child: my id is 434
Child: my id is 430, my parent has just terminated me
Child: my id is 434, my parent has just terminated me
Child: my id is 433, my parent has just terminated me
Child: my id is 432, my parent has just terminated me
Child: my id is 431, my parent has just terminated me
Parent: I have killed all of my children
```

Note: Use the system call kill() send signal from parent to child. The parent needs an array to store process id of all children. The system call getpid() is used to get the process id of the process.